Guía 1 de 4 — La tabla de pandas: elegir y filtrar

DPL1046 · Clase 5 · Para repasar en casa, a tu ritmo.

pandas es la herramienta central del curso: casi todo lo que haces con datos pasa por una tabla de pandas. Esta guía repasa qué es esa tabla y cómo **elegir columnas, filtrar filas y ordenar**, con las tablas de Olist.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado. Todos los ejemplos usan `import pandas as pd` y `RUTA = "data/raw/"`.

---

1. DataFrame y Series

Piensa en una **hoja de Excel**: filas, columnas y un nombre arriba de cada columna. Un **DataFrame** es esa hoja dentro de Python. Una **Series** es **una sola columna** de esa hoja.

Palabra	Qué es	Ejemplo
DataFrame	La tabla completa	`orders`
Series	Una columna	`orders["order_status"]`

```
import pandas as pd

RUTA = "data/raw/"
orders = pd.read_csv(RUTA + "olist_orders_dataset.csv")

print("Filas y columnas:", orders.shape)            # (99441, 8)
print("Nombres de columnas:", list(orders.columns))
print(orders.head(3))                               # las 3 primeras filas

estados = orders["order_status"]                    # una columna → Series
print(type(orders).__name__)                        # DataFrame
print(type(estados).__name__)                       # Series
print(estados.value_counts())                       # cuenta cada valor
print(orders.dtypes)                                # tipo de cada columna
```

- `shape` = (filas, columnas). **No lleva paréntesis.**
- `head(3)` = las tres primeras filas, como abrir un libro en la primera página.
- `value_counts()` = una **lista de asistencia por categoría**: cuenta cuántas veces aparece cada valor. `delivered` es lejos el más frecuente.
- `dtypes` muestra el tipo de dato. En `orders`, **todo dice `object`** (texto), incluso las fechas. Esa es la razón de que más adelante haya que convertirlas.

> Los puntos suspensivos `...` en la salida son pandas abreviando columnas que no caben en pantalla. No es un error.

---

2. Elegir columnas

```
dos_columnas = orders[["order_id", "order_status"]]
print(dos_columnas.head(3))
```

Fíjate en los **dos pares de corchetes**. El de afuera es el de siempre (“busca en la tabla”); el de adentro es una **lista** con los nombres de las columnas que quieres.

Quiero…	Escribo	Resultado
Una columna	`orders["order_status"]`	Series
Varias columnas	`orders[["order_id", "order_status"]]`	DataFrame

---

3. Filtrar filas

Un filtro son **dos pasos**:

1. Una pregunta que se responde con **verdadero o falso para cada fila**.
2. Quedarse solo con las filas verdaderas.

order_status	¿es canceled?
delivered	False
canceled	**True**
delivered	False
shipped	False

```
cancelados = orders[orders["order_status"] == "canceled"]
print("Pedidos cancelados:", len(cancelados))     # 625
```

Se lee de adentro hacia afuera: `orders["order_status"] == "canceled"` pregunta fila por fila; `orders[ ... ]` deja solo las verdaderas. Ojo con el **doble igual**: `==` **pregunta**, `=` **guarda**.

Dos condiciones a la vez

```
items = pd.read_csv(RUTA + "olist_order_items_dataset.csv")
caros = items[(items["price"] > 500) & (items["freight_value"] > 50)]
print("Ítems caros con flete alto:", len(caros))
```

Cada condición **entre paréntesis**, unidas con `&` (“y”) o `|` (“o”).

Operador	Significa	Ejemplo
`==`	igual a	`orders["order_status"] == "canceled"`
`!=`	distinto de	`orders["order_status"] != "delivered"`
`>` `<` `>=` `<=`	mayor, menor…	`items["price"] > 500`
`&`	y (las dos)	`(a > 1) & (b > 2)`
`.isin([...])`	está en la lista	`orders["order_status"].isin(["shipped", "canceled"])`

Para **“o”** se usa la barra vertical `|` (en lugar de `&`): `(a > 1) | (b > 2)`.

> **El error número uno:** usar `and` / `or` en vez de `&` / `|`, o **olvidar los paréntesis**. Si ves *“truth value of a Series is ambiguous”*, es exactamente eso.

---

4. Ordenar y tomar una fila

```
top3 = items.sort_values("price", ascending=False).head(3)
print(top3[["order_id", "price"]])

print(orders.iloc[0])      # la fila en la posición 0
```

- `sort_values("price", ascending=False)` ordena por precio **de mayor a menor**. Con `ascending=True` (o sin ese parámetro) sería de menor a mayor.
- `.head(3)` se queda con las tres primeras: los tres ítems más caros.
- `iloc[0]` busca por **posición** (no por nombre). Igual que en las listas, **se cuenta desde 0**.

> Ninguna de estas operaciones modifica el CSV original. Cada una **devuelve una tabla nueva**; si la quieres usar después, la guardas en una variable.

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`KeyError: 'order_stauts'`	Escribiste mal el nombre de la columna	Compáralo con `list(orders.columns)`
`truth value of a Series is ambiguous`	Usaste `and` / `or` o faltan paréntesis	Usa `&` / `|` y paréntesis en cada condición
`TypeError: '>' not supported between 'str' and 'int'`	Comparas texto con número	Revisa `dtypes` de la columna
El filtro devuelve 0 filas	El valor está mal escrito (mayúsculas, tildes)	Mira los valores reales con `value_counts()`
`KeyError` al usar dos columnas	Usaste un solo corchete	Varias columnas = **doble corchete**

Practica tú

1. ¿Cuántos pedidos hay con estado `shipped`?
2. Muestra `order_id` y `price` de los ítems con precio menor a 5.
3. Muestra los 5 ítems **más baratos**.
4. ¿Cuántos pedidos están `canceled` **o** `unavailable`?

▼ **Ver respuestas**

1. `len(orders[orders["order_status"] == "shipped"])`
2. `items[items["price"] < 5][["order_id", "price"]]`
3. `items.sort_values("price").head(5)`
4. `len(orders[orders["order_status"].isin(["canceled", "unavailable"])])`

Para llevarte

- **DataFrame** = tabla; **Series** = columna.
- Una columna: un corchete. **Varias: doble corchete.**
- Un filtro = pregunta verdadero/falso por fila + quedarse con las verdaderas.
- Condiciones: **paréntesis y `&` / `|`**.
- Nada de esto cambia el archivo original.